

Canberra SmartWaste: A Smart Waste Management System for Canberra

Group Members: Bijay Pokhrel, Samir Bhandari, Krishna Trivedi, Ayush Ale, Charanpal Kaur

GitHub Repository: <https://github.com/charanpalkaur06-coder/Smart-waste-management-system>

1. Project Overview

Canberra SmartWaste is an Iteration 1 prototype to make it easier to see and manage municipal waste collection in Canberra. Efficiencies can be lost with fixed collection days due to the fact that bins in busy areas might fill at different times, and refuse collection vehicles could be taking out bins that don't need to be emptied. As the number of smart-waste studies grows, they are employing a variety of sensor data, central monitoring and operational decision support to enable a more responsive collection. (Sosunova & Porras, 2022). In the same vein, in an Australian municipal context An et al. (2022) prove how smart-bin fullness data can be used to complement a dashboard for decision-makers.

So the goal of the Iteration 1 was to develop the software prototype and not a deployment of Internet of Things (IoT). This system is based on simulated Canberra smart bin data, and offers 3 role based experiences: Citizen, Collection Staff and Administrator. Prototyping, citizen waste-issue reporting and tracking, smart-bin monitoring, a reusable fill status algorithm, a deterministic collection priority scoring, a suggested collection sequence, collection completion, dashboard statistics, search and alerts, maintenance diagnostics, and local persistence and automated testing completed work. Readable WST-2026-XXXX reference numbers are provided for citizens and they can track the report status along Submitted, Under Review, Scheduled and Resolved.

Recent literature has demonstrated that monitoring and sharing information in smart bins can help in the municipal maintenance, collection and routing decisions (Wang et al., 2021). Public participation is also relevant because the outcomes of waste management also rely on partial participation of the residents in the local waste practices (Kuang & Lin, 2021). As a result, Canberra SmartWaste is an operational monitoring tool, along with a citizen reporting tool.

2. System Implementation

2.1 Architecture and Software Components

The prototype is done using React 19, TypeScript 6 and Vite 8. The choice for React was made due to the support for re-usable dashboards, tables, forms, alerts and role-specific views. For users, bins, reports, collection records, status values and priorities don't have to have the same domain type, which helps to avoid errors from having inconsistent values. We use modular CSS to style and interface icons are provided by Lucide React and data visualisation by Recharts.

React Context is used to handle the shared application state. AppContext handles the authenticated role, bins, reports and collections and provides actions for log in, log out, submission of reports, changes of report status, assignment of routes, completion of collections and reset of demonstration data. The persistence is centralised in `dataStore.ts` with the help of `localStorage` available in the browser. This was made as an engineering choice in Iteration 1 designed to ensure browser persistence and to enable an repeatable demonstration in the classroom without the need to add a backend that wasn't necessary for the first prototype. Architecture continues to be extensible as the persistence service may easily be replaced with API and database calls in the future.

One significant architecture improvement that was made was to eliminate duplicate run-time data sources. The dashboard, map, alerts and collection views now share the current state of the application. Accordingly, dependent views will be updated regularly, and not continue to show stale seed data, when the fill level of the bin changes during collection.

2.2 User Interface and Functional Modules

The authentication of a prototype starts in an unauthenticated mode. Users log in with fake test accounts and the app checks if the account is the one they have selected to log in. The Citizen, Collection Staff and Administrator functionality are separated by role controls. Citizens can view reporting and report history; collection staff can view the required pickup work; and the administrators can view the operational dashboard, public-report administration, smart-bin views, suggested collection sequence, diagnostics and reset demonstration-data.

The citizen reporting module includes issue category, location, Canberra suburb, urgency, waste type, optional bin identification, description and optional contact information. With validation, you cannot leave out required data and white

space descriptions. Upon successful submission, a unique reference, WST-2026-XXXX is assigned, and a Submitted status is set. The same record is accessible to administrators, and can be advanced through several statuses: Under Review, Scheduled and Resolved. This common workflow gives the traceability between the public input and operational action. Kuang and Lin (2021) highlight supporting facilities and contextual factors as affecting public involvement in waste practices and thus the need to ensure that there is a visible reporting interface through digital technology alongside the internal municipal process.

The simulated data is from a number of locations around Canberra, such as Belconnen, Gungahlin, Woden, Dickson, Kingston and Canberra City. The `getBinStatus()` function can be called repeatedly as a re-usable function and falls into four predetermined status bins: 0–49% Normal, 50–79% Moderate, 80–89% Collection Required and 90–100% Critical. Values put in to the input are securely limited to stay away from errors due to inappropriate sensor values. The thresholds are project specific business rules and not industry standards claimed. This logic is used in the map, dashboard and collection screens as well to keep the classifications consistent.

The fill contribution, report urgency and time since collection are factors in the collection-priority algorithm. Sources of fill level is worth up to 50 points, urgency up to 30 points and overdue time up to 20 points. The final score falls into one of the levels Low, Medium, High or Critical. This is not a black box A.I. based approach, but a rule based approach that is transparent. Canberra SmartWaste provides an example of the same general principle that is demonstrated by Roy et al. (2022) and Ghahramani et al. (2022) on an achievable level of Iteration 1.

The required collection sequence only includes bins that are either Collection Required or Critical. The sequence can be assigned to collection staff and the assignment information is stored in the data layer. When staff put a bin as collected, the fill level will be reset to 5%, it will become a Normal bin, last collected time will be updated, priority will be recalculated, related collection records will be set as Completed, and all the reports that are not explicitly connected to this bin will be not be automatically resolved. This is an explicit relationship that will stop unrelated reports from the same suburb from being incorrectly resolved.

The administrator dashboard is populated with application state

statistics including monitored bins, critical bins, bins for collection, active reports, resolved reports and average network fill. Dynamic alerts help maximize the efficiency of hazardous bin and new reports and pending collections identification. The ability to search includes bins, reports and collection records. There is also a maintenance view that shows simulated sensor faults and can enable prototype maintenance-ticket acknowledgement. These qualities all illustrate integration among modules, NOT a series of standalone interface screens.

2.3 Testing and Quality Controls

Vitest unit tests validate business-critical logic, such as fill-level boundaries, invalid sensor values, status normalization, priority calculation, report validation, report-reference generation and report-resolution relationships. The difference between testing and a verification is that testing is used to make sure that the collection completion addresses a report that is explicitly listed for addressing the collection completion and it does not address a different report. ESLint, and a production build process are also part of the project. These controls enhance maintainability and minimize the risk of integration changes to disrupt the key demonstration workflows.

3. Technical Challenges

Maintaining state consistency was a big technical challenge. Previous development was done with the separation of seed or mock data from the active application state, allowing an update to one screen while another had an out of date value. The answer was to have a single view of runtime state via AppContext and the data service in order to access the same information for dashboard stats, map markers, alerts and collection views. This was an affirmation of the software-engineering concept of having one source of truth in stateful applications.

The second challenge was to standardise the business states. Values were replaced or normalised as per the following for new, assigned and resolved respectively: Submitted, Under Review and Scheduled and Resolved, and collection states were standardised separately. No ambiguity with the TypeScript union types and normalisation helpers. Consistency work was done to match the collection threshold and so that Moderate bins do not end up on the wrong side of the threshold as required pickups.

The third challenge was related to the relationship between reports and collections. If all of the same suburb's reports were automatically resolved, when the bin was emptied, then

inaccurate data would result. The final implementation thus only solves those reports where the binId is exactly the same as the one collected. This change helped to ensure the integrity of the data and it also created a good automation testing scenario.

The team also had to keep in mind technical ambition and scope of Iteration 1. Advanced IoT sensing and vehicle-routing optimisation has been shown, however, and without the use of physical sensors, or a production backend, this would have resulted in false claims. The final prototype is thus a labeled simulated telemetry with suggested sequence of priorities instead of the shortest-path or AI optimised telemetry. One of the main takeaways from Iteration 1 was about how important it is to represent accurately what the software actually implements in order to achieve technical credibility.

4. GitHub Repository and Team Contributions

Source-control and evidence of technical contribution was kept in GitHub. The repository of Canberra SmartWaste is: charanpalkaur06-coder / Smart-waste-management-system. The branching strategy involves three main branches: main - the stable release branch, develop - the integration branch, and feature branches for individual features that will be developed. This will make it less likely to break the demonstration build and will make it easier to check out the contribution history at each module level.

The major branches are bijay-admin-dashboard, samir-citizen-interface, krishna-database-collection, ayush-auth-testing and charanpal-maintenance-analytics. Bijay worked on the admin/dashboard functionality and project coordination, Samir on the Citizen Reporting interface and validation, Krishna on the data models and persistence and algorithms for data collection, Ayush on authentication, shared-state integration and testing, and Charanpal on smart bin sensor diagnostics and maintenance functionality. Commit history has entries for changes in these areas under Commit, Correction and testing activities, and the changes are successfully merged into develop and then into main.

Coordinating day-to-day communication and sharing of resources was done via Google Chat, while GitHub was the definitive code and version-control history. This separation guaranteed that there was no informal communication instead of the evidence of development that was traceable.

5. Jira Project Management

Jira is the formal record of the task-management during the Iteration 1. The backlog is structured in the same way as the work packages are at GitHub: authentication and access control, citizen reporting, data and business algorithms, administrator monitoring, collection management, maintenance and diagnostics, integration testing, and assessment preparation. Issues are logged and clearly described, prioritised and have acceptance criteria as well as workflow status set to the member who is responsible for the module.

During the Iteration 1 sprint, there is a separation between To Do and In Progress and Done work in order to allow a review of the work being done on the project, instead of describing it in the final report. Current development issues are role-based prototype login, citizen reporting, status of the reports, central persistence service, classification of the smart bins, collection priority logic, dashboard integration, map and alert synchronisation, collection execution and maintenance acknowledgement and core automated tests.

The other assessment related activities are report finalisation, preparing presentation and rehearsing for Jira evidence capture and demonstration. Having these tasks all in one board gives you a good overview of completed milestones, and of outstanding work. It also enables the teaching team to map the ownership of the tasks in Jira to the branches and contributions done in GitHub, thus giving traceability between the planning, implementation, and delivered software.

6. Demonstration, Completed Milestones and Remaining Work

Week 6 will NOT be a series of separate screens, but will be an end-to-end workflow. Each of the 5 team members will have 2 minutes to present with 5 minutes for questions after the 10 minute presentation. The demonstration includes unauthenticated access, citizen report submission and tracking, administrator review, smart-bin classification and priority logic, collection assignment, staff collection completion, dashboard/synchronisation with map/alerts, maintenance diagnostics and reset of demonstration data.

Iteration 1 Milestones are achieved at prototype: Three roles, persistent demonstration data, citizen to administrator flow of reports, deterministic collection decisions, collection execution and integrated monitoring. The following versions should include real telemetry and/or API-based IoT data, a backend

database and production grade authentication, real geospatial mapping and notifications, and explore advanced vehicle-routing optimisation. The above-mentioned future directions are in line with the latest research on smart waste systems using IoT technology and dynamic waste collection (Addas et al., 2024; Hess et al., 2024), and are definitely not covered by the prototype mentioned in Iteration 1.

□

References

- Addas, A., Khan, M. N., & Naseer, F. (2024). Waste management 2.0 leveraging internet of things for an efficient and eco-friendly smart city solution. *PLOS ONE*, 19(7), e0307608. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0307608>
- An, Y., Qiu, J., & Dong, Z. Y. (2022). Smart waste management system for decision makers by using smart bins: A case study for an Australian municipality. *Australian Journal of Civil Engineering*, 20(2), 454–459. <https://www.tandfonline.com/doi/full/10.1080/14488353.2021.2024329>
- Ghahramani, M., Zhou, M., Molter, A., & Pilla, F. (2022). IoT-based route recommendation for an intelligent waste management system. *IEEE Internet of Things Journal*, 9(14), 11883–11892. <https://arxiv.org/abs/2201.00180>
- Hess, C., Dragomir, A. G., Doerner, K. F., & Vigo, D. (2024). Waste collection routing: A survey on problems and methods. *Central European Journal of Operations Research*, 32, 399–434. <https://link.springer.com/article/10.1007/s10100-023-00892-y>
- Kuang, Y., & Lin, B. (2021). Public participation and city sustainability: Evidence from urban garbage classification in China. *Sustainable Cities and Society*, 67, 102741. <https://doi.org/10.1016/j.scs.2021.102741>
- Roy, A., Manna, A., Kim, J., & Moon, I. (2022). IoT-based smart bin allocation and vehicle routing in solid waste management: A case study in South Korea. *Computers & Industrial Engineering*, 171, 108457. <https://www.sciencedirect.com/science/article/abs/pii/S0360835222004910>
- Sosunova, I., & Porras, J. (2022). IoT-enabled smart waste

management systems for smart cities: A systematic review. *IEEE Access*, 10, 73326–73363.

<https://ieeexplore.ieee.org/document/9815071/>

Wang, C., Qin, J., Qu, C., Ran, X., Liu, C., & Chen, B. (2021). A smart municipal waste management system based on deep-learning and Internet of Things. *Waste Management*, 135, 20–29. <https://pubmed.ncbi.nlm.nih.gov/34461487/>